

Résumé de cours : Python pour la Data Science

Formateur : ALIMOU DIALLO • Version du 27/10/2025

Objectif : fournir une fiche complète et pratique pour importer, explorer, nettoyer, visualiser et préparer des données avec Python (pandas, NumPy, scikit-learn, matplotlib, seaborn).

1. Importation des données

Avant toute manipulation (nettoyage, transformation, visualisation), il faut importer les données avec la bibliothèque pandas, référence pour la donnée tabulaire (CSV, Excel, JSON, SQL, etc.).

Exemples CSV / Excel / JSON / SQL

```
import pandas as pd

# CSV
df = pd.read_csv("data.csv")          # encoding="utf-8", sep=";",
decimal=",", na_values=["", "NA"]
df.head()

# Excel (première feuille par défaut)
df_xlsx = pd.read_excel("data.xlsx", sheet_name=0)

# JSON (enregistrement ligne par ligne)
df_json = pd.read_json("data.json", lines=True)

# SQL (ex: SQLite)
import sqlite3
conn = sqlite3.connect("database.sqlite")
df_sql = pd.read_sql_query("SELECT * FROM table_clients", conn)
```

Astuces :

- Spécifier les options de lecture (sep, encoding, decimal, na_values) selon la source.
- Toujours visualiser quelques lignes pour vérifier l'import (head, sample).

2. Vérification et exploration des données

Comprendre la structure du jeu de données : dimensions, types, valeurs manquantes, doublons, statistiques descriptives.

```
# Dimensions et aperçu
df.head()          # 5 premières lignes
df.tail()          # 5 dernières lignes
df.shape           # (nb_lignes, nb_colonnes)
len(df)            # nombre de lignes

# Types et informations
df.dtypes
df.info()

# Valeurs manquantes
df.isnull().sum()

# Statistiques descriptives
df.describe(include="all") # inclut numériques + catégorielles

# Doublons
df.duplicated().sum()
df = df.drop_duplicates()
```

Étape	Objectif	Outils principaux
Importation	Charger les données	pd.read_csv(), read_excel(), read_sql()
Vérification	Comprendre la structure	head(), info(), describe()
Nettoyage	Gérer NA & doublons	dropna(), fillna(), drop_duplicates()
Outliers	Identifier & traiter	boxplot(), IQR
Scaling	Mettre à l'échelle	MinMaxScaler, StandardScaler
Visualisation	Explorer & communiquer	matplotlib, seaborn, pandas.plot()

3. Data Cleaning : valeurs manquantes

La suppression est un dernier recours. Préférer l'imputation (moyenne, médiane, mode) selon le type de variable et la distribution.

```
# Suppression (à utiliser prudemment)
df = df.dropna() # axis=0 (lignes) ou axis=1 (colonnes)

# Imputation simple
df['age'] = df['age'].fillna(df['age'].median()) # numérique
df['ville'] = df['ville'].fillna(df['ville'].mode()[0]) # catégorielle
```

```
# Imputation groupée (par catégorie, ex: 'sexe')
df['revenu'] = df.groupby('sexe')['revenu'].transform(lambda s:
s.fillna(s.median()))
```

Gestion des incohérences

```
# Remplacer des libellés incohérents
df['ville'] = (df['ville']
               .str.strip()
               .str.title()
               .replace({'Pariss': 'Paris', 'Lyonn': 'Lyon'}))

# Convertir des types
df['date'] = pd.to_datetime(df['date'], errors='coerce')
df['code_postal'] = df['code_postal'].astype('string')
```

4. Valeurs aberrantes (Outliers)

Détecter via BoxPlot et traiter selon le contexte (erreur de saisie vs valeur extrême légitime).

```
import numpy as np

# Détection via IQR
q1 = df['montant'].quantile(0.25)
q3 = df['montant'].quantile(0.75)
iqr = q3 - q1
borne_inf = q1 - 1.5 * iqr
borne_sup = q3 + 1.5 * iqr

# Filtrage
df_filtre = df[(df['montant'] >= borne_inf) & (df['montant'] <=
borne_sup)]

# Option: winsorisation (clip)
df['montant_clip'] = df['montant'].clip(lower=borne_inf,
upper=borne_sup)
```

5. Feature Scaling : normalisation & standardisation

Mettre les variables sur la même échelle pour les modèles sensibles aux distances (k-NN, SVM, régression logistique, PCA, etc.).

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Min-Max scaling -> [0, 1]
mm = MinMaxScaler()
X_mm = mm.fit_transform(df.select_dtypes('number'))
```

```
# Standardisation (Z-score): moyenne=0, écart-type=1
ss = StandardScaler()
X_ss = ss.fit_transform(df.select_dtypes('number'))
```

6. Visualisation des données

La visualisation permet d'explorer, de vérifier des hypothèses et de communiquer des résultats. Utiliser principalement matplotlib, seaborn et pandas.plot().

Principes de base

- Choisir un graphique adapté (barres, lignes, histogrammes, boxplots, nuages de points).
- Toujours ajouter un titre, des labels d'axes et une légende si nécessaire.
- Limiter le nombre de couleurs et éviter les éléments superflus (grid simple).
- Utiliser des agrégations et groupby pour synthétiser l'information.

Matplotlib (basique)

```
import matplotlib.pyplot as plt

# Courbe temporelle
plt.figure()
df.set_index('date')['ventes'].plot(title='Ventes dans le temps')
plt.xlabel('Date')
plt.ylabel('Ventes')
plt.tight_layout()
plt.show()

# Histogramme
plt.figure()
df['montant'].plot(kind='hist', bins=30, title='Distribution du
montant')
plt.xlabel('Montant')
plt.tight_layout()
plt.show()
```

Seaborn (exploration rapide)

```
import seaborn as sns

# Nuage de points
sns.scatterplot(data=df, x='revenu', y='dépenses', hue='sexe')

# Boxplot par groupe
sns.boxplot(data=df, x='sexe', y='revenu')

# Heatmap de corrélation
corr = df.select_dtypes('number').corr()
sns.heatmap(corr, annot=True)
```

pandas.plot() (API pratique)

```
# Barres par catégorie
(df.groupby('categorie')['montant']
 .sum()
 .sort_values(ascending=False)
 .plot(kind='bar', title='Montant total par catégorie'))

# Plusieurs séries
df[['revenu', 'dépenses']].plot(kind='line', title='Revenu vs Dépenses')
```

Sauvegarder les graphiques

```
plt.savefig('figure_ventes.png', dpi=300, bbox_inches='tight') # avant
plt.show()
```

7. Workflow type (de l'import au graphique)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler

# 1) Import
df = pd.read_csv('data.csv')

# 2) Vérifications
print(df.shape, df.dtypes)
print(df.isna().sum())

# 3) Nettoyage (exemples)
df = df.drop_duplicates()
df['age'] = df['age'].fillna(df['age'].median())

# 4) Outliers
q1 = df['revenu'].quantile(0.25); q3 = df['revenu'].quantile(0.75)
iqr = q3 - q1; low = q1 - 1.5*iqr; high = q3 + 1.5*iqr
df = df[(df['revenu'] >= low) & (df['revenu'] <= high)]

# 5) Scaling
scaler = StandardScaler()
cols_num = df.select_dtypes('number').columns
df[cols_num] = scaler.fit_transform(df[cols_num])

# 6) Visualisation
sns.boxplot(data=df, x='sexe', y='revenu')
plt.title('Revenu standardisé par sexe')
plt.tight_layout()
plt.show()
```

8. Bonnes pratiques & pièges courants

- Toujours vérifier l'encodage et le séparateur des fichiers CSV.
- Documenter chaque transformation (journal de nettoyage).
- Séparer données d'entraînement / test avant tout data leakage.
- Traiter les NA et les outliers avec prudence (garder une copie brute).
- Sauvegarder les figures avec des résolutions suffisantes (dpi $\geq$ 300).